TECH MEETUP · 2026

Creación de Agentes para Mortales

Sin doctorado. Sin miedo. Con criterio.



GitHub Copilot · Agent Mode · Skills · MCP · Diseño responsable

De qué vamos a hablar hoy

01
¿Qué es un agente?
Y qué NO es

02
El error de añadir más agentes
La trampa del multi-agente

03
3 patrones que funcionan
Probados en producción

04
Agente vs Skill vs Edits
Cuándo usar qué

05
GitHub Copilot en práctica
Agent Mode, MCP, .agent.md

06
Diseño responsable
Criterio antes que código

01 · ¿Qué es un agente?



Un agente es un modelo de IA que puede razonar, planificar y usar herramientas para completar tareas de forma autónoma — sin que tú ejecutes cada paso.



Razona

Entiende el objetivo y decide los pasos



Usa tools

Llama a APIs, lee archivos, ejecutacódigo

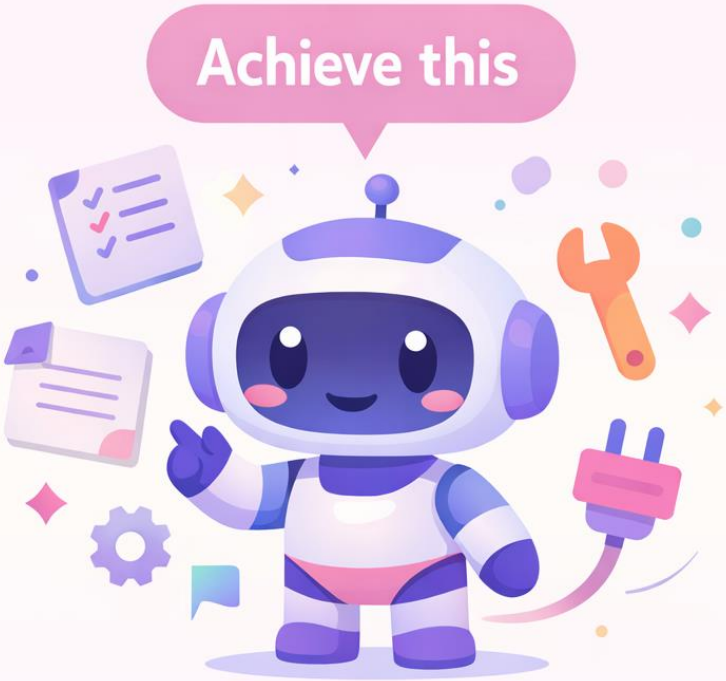


Itera

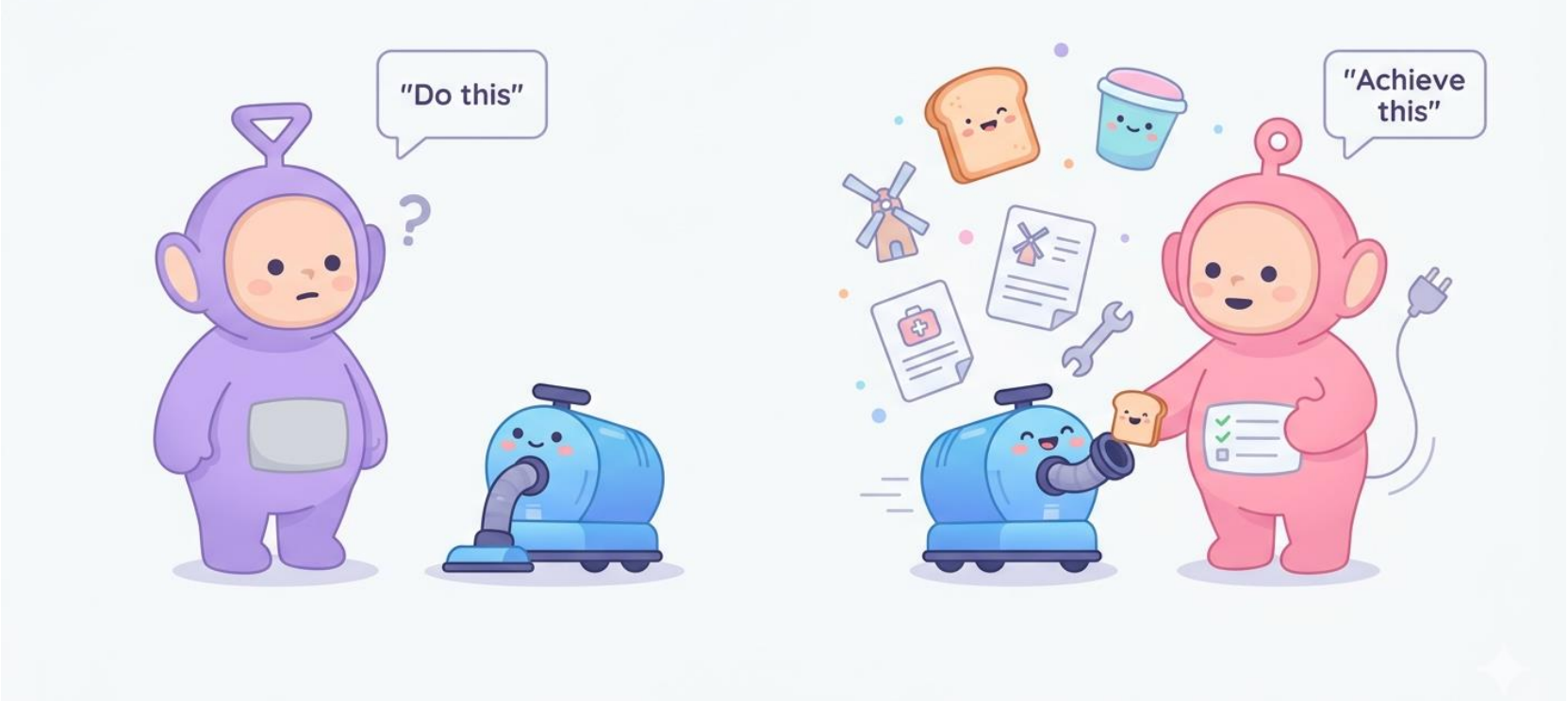
Revisa su propio output y corrige errores

Un agente es un modelo de IA que puede razonar, planificar y usar herramientas para completar tareas de forma autónoma — sin que tú ejecutes cada paso.





Un agente es un modelo de IA que puede razonar, planificar y usar herramientas para completar tareas de forma autónoma — sin que tú ejecutes cada paso.



02 · La trampa del multi-agente



17x

más errores en redes
de agentes sin estructura

Google DeepMind, dic. 2025 · 180 configs, 5 arquitecturas



Gartner: +40% de proyectos agenticos
se cancelarán antes de 2027



El error compuesto es matemático:
10 pasos al 95% = 60% éxito total



Saturación en 4 agentes: añadir
más no mejora, solo sobrecarga

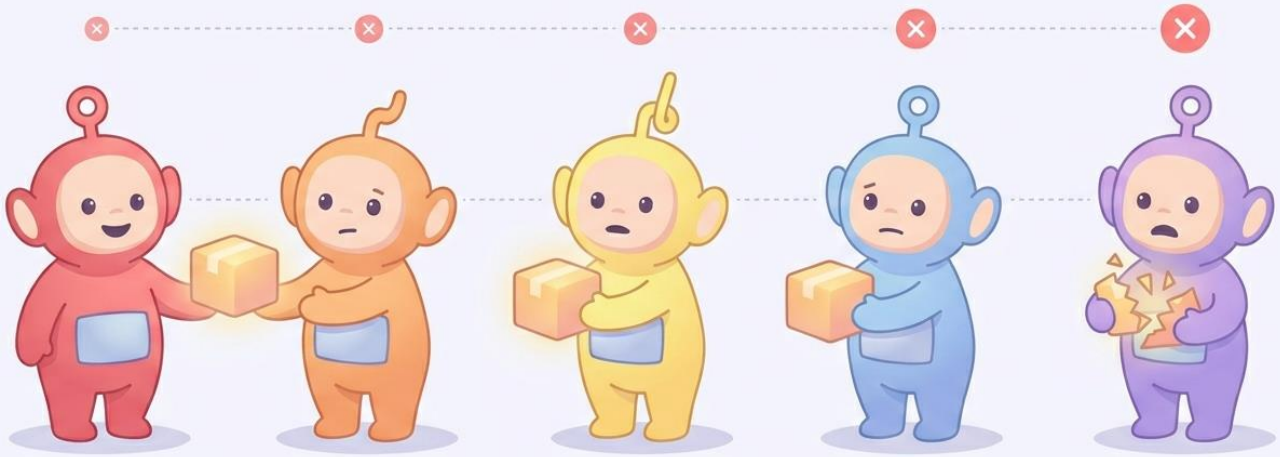
El error compuesto es matemático:
10 pasos al 95% = 60% éxito total



Errors compound

El error compuesto es matemático:
10 pasos al 95% = 60% éxito total

95% × 95% × 95% × 95% × 95% = 77%



Errors compound

03 · 3 patrones que funcionan en producción

Plan & Execute

Cómo funciona

Un modelo caro planifica. Modelos baratos ejecutan.

Cuándo usarlo

Tareas secuenciales claras. Klarna usa esto.

⚠ Falla si el entorno cambia a mitad del plan

Supervisor / Worker

Cómo funciona

Un supervisor decide y delega. Workers se especializan.

Cuándo usarlo

Tareas heterogéneas que requieren especialización.

⚠ El supervisor puede convertirse en cuello de botella

Swarm (Handoffs)

Cómo funciona

Sin supervisor. Agentes se pasan el contexto entre ellos.

Cuándo usarlo

Flujos de alto volumen y bien definidos. Ej: soporte.

⚠ Sin observabilidad, depurar es una pesadilla

03 · PATRÓN 1 DE 3

Plan & Execute



Cómo funciona

Un modelo potente analiza el objetivo y lo descompone en pasos concretos. Modelos más baratos ejecutan cada paso sin tener que razonar el problema entero.

Cuándo usarlo

Tareas con objetivo claro que se pueden secuenciar: procesado de documentos, pipelines de datos, flujos de soporte. Klarna lo usa para bajar de 11 min a menos de 2 min por ticket.

⚠ Riesgo a tener en cuenta

Si el entorno cambia a mitad de la ejecución, el plan inicial queda obsoleto. En tareas volátiles, añadir puntos de re-planificación o cambiar de patrón.

Supervisor / Worker



Cómo funciona

Un agente supervisor enruta cada tarea al worker especializado. Si hay conflicto entre decisiones, el supervisor actúa como árbitro único. Evita que dos agentes tomen decisiones contradictorias al mismo tiempo.

Cuándo usarlo

Tareas heterogéneas donde distintos tipos de input requieren respuestas muy distintas. Soporte con escalación, pipelines de revisión con múltiples criterios, análisis financiero de varias fuentes.

⚠ Riesgo a tener en cuenta

Si el supervisor aprueba cada microdecisión se convierte en cuello de botella. Solución: autonomía acotada por worker, escalar solo los casos que realmente lo necesiten.

Supervisor / Worker



Cómo funciona

Un agente supervisor enruta cada tarea al worker especializado. Si hay conflicto entre decisiones, el supervisor actúa como árbitro único. Evita que dos agentes tomen decisiones contradictorias al mismo tiempo.

Cuándo usarlo

Tareas heterogéneas donde distintos tipos de input requieren respuestas muy distintas. Soporte con escalación, pipelines de revisión con múltiples criterios, análisis financiero de varias fuentes.

⚠ Riesgo a tener en cuenta

Si el supervisor aprueba cada microdecisión se convierte en cuello de botella. Solución: autonomía acotada por worker, escalar solo los casos que realmente lo necesiten.

03 · PATRÓN 3 DE 3

Swarm · Handoffs entre agentes

Sin supervisor centralizado · cada agente declara sus rutas de handoff



✓ Flujo de alto volumen resuelto

Sin trazabilidad completa de cada paso, depurar por qué el usuario llegó al Agente C en lugar del Agente B es una pesadilla.
No desplegar sin observabilidad.

Cómo funciona

No hay supervisor. Cada agente pasa el contexto completo al siguiente según lo que detecta en la tarea. Las rutas de handoff están declaradas de antemano — el sistema fuerza que se respeten.

Cuándo usarlo

Flujos de alto volumen, bien definidos, con pocas excepciones. Soporte al cliente, onboarding multi-paso, sistemas de triage. La lógica de enrutamiento debe ser clara desde el principio.

⚠ Riesgo crítico

Es el patrón más difícil de depurar. Sin logs y trazabilidad por paso desde el día 1, encontrar fallos en producción puede llevar días. Si no tenéis observabilidad, elegid otro patrón.

04 · ¿Agente, Skill o Edits?

	Inline / Edits	Skill	Agent Mode
Cuándo usarlo	Sugerencias rápidas, completado de código	Tarea repetible y bien definida	Tarea compleja, multi-paso, ambigua
Autonomía	Tú ejecutas cada paso	Automatiza la tarea dentro de su scope	Planifica y ejecuta solo, itera si falla
Riesgo	**Muy bajo**	**Bajo (scope acotado)**	**Alto si no se revisa**
Ejemplo	Autocompletar función en VS Code	Skill que genera comentarios de código	Agent que toma un issue y hace PR

05 · GitHub Copilot en práctica



Agent Mode

› Loop autónomo: razona, ejecuta tools, itera

› Perfectopara tareas multi-archivo y multi-paso

› Revisar siempre el output antes de aceptar



Archivo .agent.md

› Define el comportamiento y scope del agente

› Puedes especificar herramientas habilitadas

› Controla handoffs entre agentes custom



MCP Connections

› Conecta Copilot a tus servicios (APIs, DBs)

› El agente puede leer y escribir datos reales

› Configurar con .mcp.json en tu workspace



Skills

› Capacidades reutilizables definidas en SKILL.md

› El agente las descubre y usa según el contexto

› Mantenerlas pequeñas y con scope muy claro

06 · Diseño responsable

Sí hacer

- Empezar con 1 agente antes de orquestar varios
- Definir scope acotado antes de codificar
- Revisar siempre el output del agente
- Usar observabilidad desde el día 1
- Documentar instrucciones en .agent.md y SKILL.md

NO hacer

- Desplegar sin revisar lo que el agente propone
- Conectar MCP a producción sin permisos mínimos
- Añadir agentes porque 'parece más potente'
- Confundir Edits con Agent Mode (distintas herramientas)
- Ignorar los costos de tokens al encadenar agentes

Sí hacer

- Empezar con 1 agente antes de orquestar varios
- Definir scope acotado antes de codificar
- Revisar siempre el output del agente
- Usar observabilidad desde el día 1
- Documentar instrucciones en .agent.md y SKILL.md

Checklist antes de desplegar tu agente



Diseño

¿Está el scope del agente claramente definido?



Infraestructura

¿Tienes observabilidad y logs de cada paso?



Calidad

¿Revisaste manualmente el primer output?



Seguridad

¿Los permisos de MCP siguen el principio de mínimo privilegio?



Resiliencia

¿Tienes un plan si el agente falla a mitad de tarea?



Costos

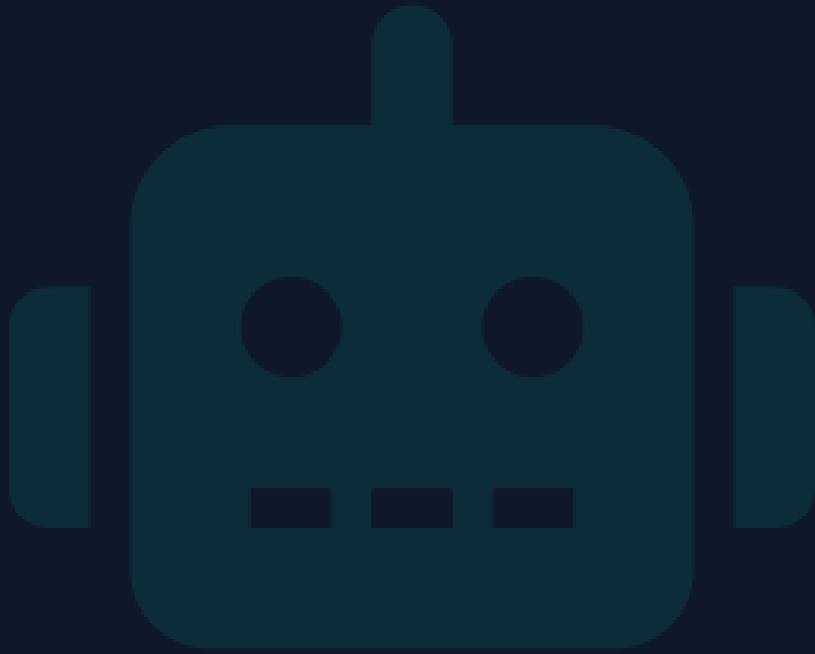
¿Calculaste el costo de tokens en producción?



Gobernanza

¿Tienes un humano que revisa antes de ejecutar pasos irreversibles?

Un agente potente no es el que hace más.



Es el que falla de forma predecible,

se puede auditar

y tiene a un humano como freno de emergencia.

¿Preguntas?

Diagrams for agent building

GitHub Copilot

Custom Agent Architecture as Code

Composable - Declarative - GitOps-deployed

5 decoupled components | GitOps lifecycle | Enterprise governance

The Composable Stack

5 declarative modules

each answering one question

The 5 Components

Separation of who | how | what rules apply



Component	File	Key question
ID **Identity** — WHO — Declarative module	.agent.md	Who am I?
SK **Capabilities** — HOW — Declarative module	SKILL.md + MCP	How do I do it?
CX **Context gates** — RULE — Declarative module	.instructions.md	What rules apply?
PR **Task templates** — TASK — Declarative module	.prompt.md	What task do I run?
EV **Event hooks** — WHEN — Declarative module	hooks.json	When do I trigger?

What each component does

ID

.agent.md

Identity

Defines the agent's name, purpose, tone and reasoning style. The entry point Copilot reads to know who it's working with.

SK

SKILL.md + MCP

Capabilities

Describes concrete skills: commands, API calls, scripts. MCP servers connect the agent to external tools in real time.

CX

.instructions.md

Context gates

Applies rules automatically by file path or domain. No massive system prompts needed - context injection is surgical.

PR

.prompt.md

Task templates

Predefined templates for recurring tasks. Team members invoke them from chat, standardizing workflows across the org.

EV

hooks.json

Event hooks

Event-driven automation on file open, commit, or PR. The bridge between agent behavior and the developer workflow.

GitOps for AI Lifecycle

Declarative flow

from definition to invocation

GitOps Lifecycle

4 phases from markdown to context-aware activation

01 DEF

Definition

YAML / MD

Pure declarative config - no code compilation, no build pipeline. Markdown + YAML frontmatter is the entire surface.

02 DIS

Discovery

MCP / Catalog

Federated at org-level. Agents are discoverable via MCP servers or the awesome-copilot catalog without manual sharing.

03 INS

Installation

Commit / Plugin

Repo commit OR plugin install. One command: copilot plugin install compliance@awesome-copilot.

04 INV

Invocation

Context-aware

Activated automatically on file open/event match, or explicitly via @agent call in the Copilot chat.

Like Homebrew distributes packages - but for AI behavior patterns. Version-controlled, code-reviewed, GitOps-deployed.

Enterprise Value

& Governance

Why this matters beyond "vibe coding"

Enterprise Value

Three pillars that move agents from prototype to production

VC — Version control

Audit trail for reasoning

- Review agent definitions like Terraform scripts
- Full git history of AI behavior changes
- Infrastructure-as-Code for intelligence
- Rollback bad agent behavior instantly

DC — Decoupled capabilities

Write once, run anywhere

- Same skill in VS Code, Claude Code, CCA
- MCP as the portability standard
- No vendor lock-in for agent logic
- Reuse skills across projects and teams

EG — Enterprise governance

Compliance built-in

- Enforce security via .instructions.md
- Rules the LLM cannot override or ignore
- Org-level policy distribution at scale
- Audit-ready by design, not afterthought

From "vibe coding"

to production agentic systems.

Version-controlled

Code-reviewed

GitOps-deployed

github.com/github/awesome-copilot | github.com/microsoft/skills

Questions?